

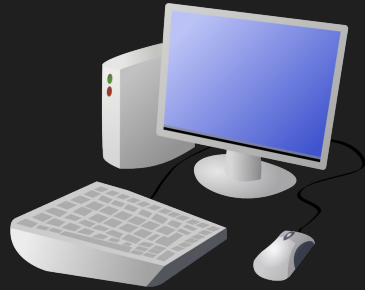
Josh Hutton

Raft Consensus in ResilientDB

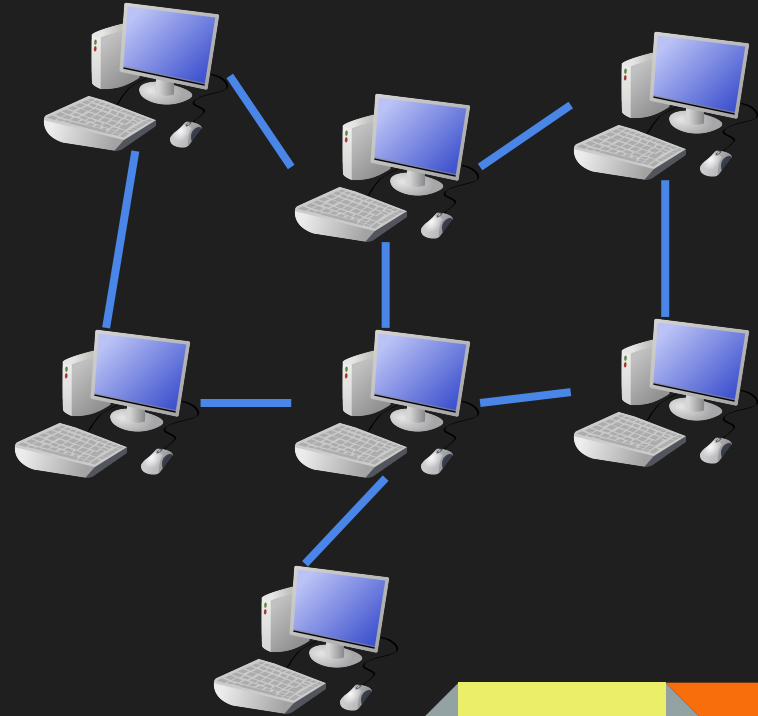


What is Consensus?

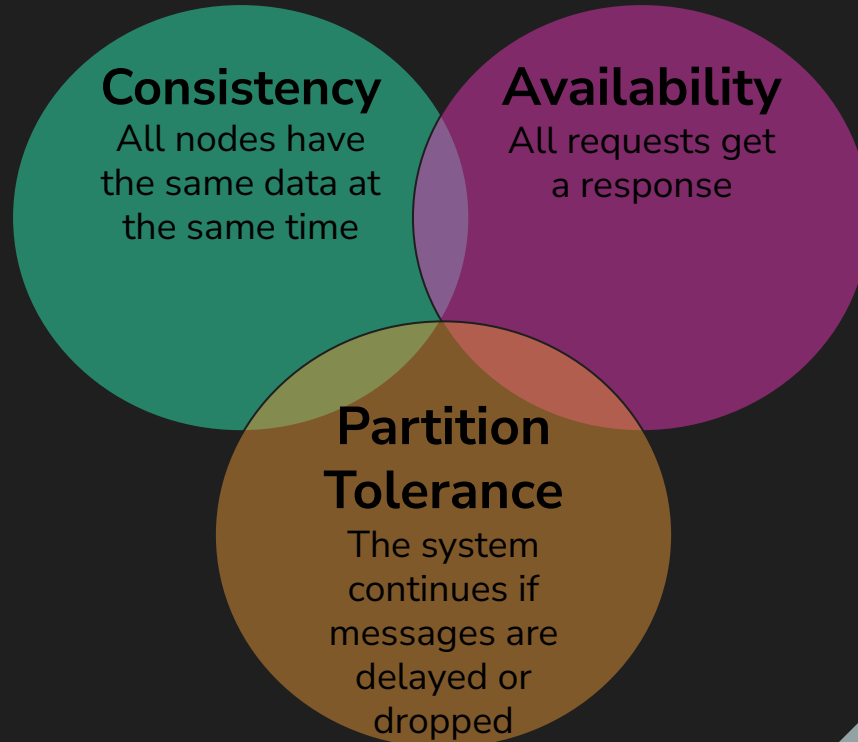
**Why do we need a consensus
algorithm at all?**



Vs



How do we approach consensus effectively?



Gilbert and Lynch, "Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services"

Raft Principles

Easy to Understand

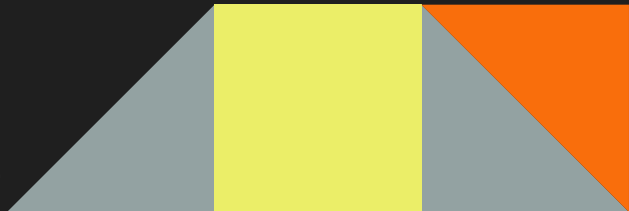
Efficient Performance $O(n)$

Crash Fault Tolerant
(CFT)

Crash, not evil

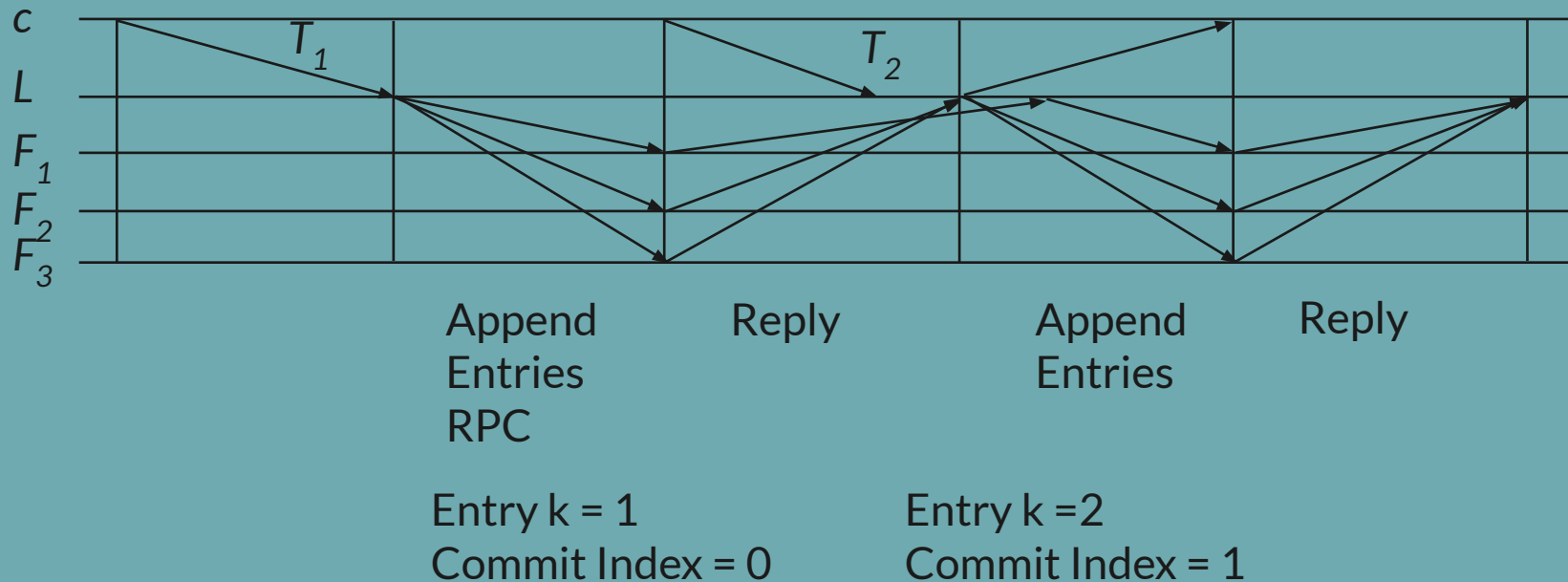
f: number of faulty nodes
Total nodes $\geq 2f+1$

Computer = Replica = Node = Machine

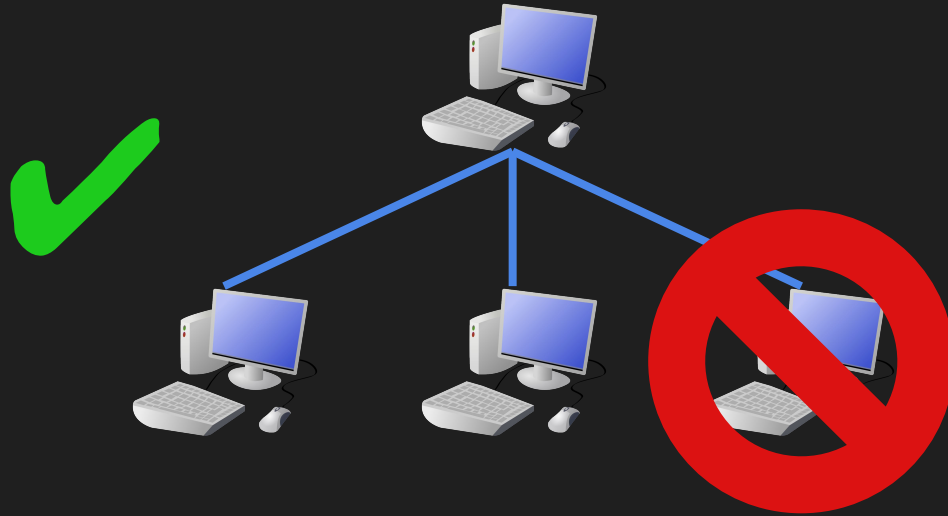


$$n > 2f + 1$$
$$n = 4$$

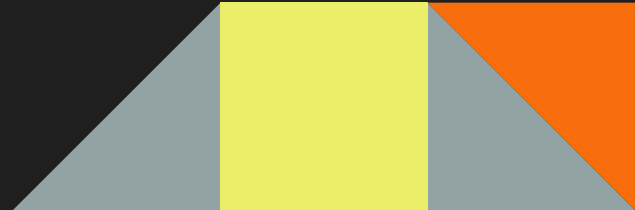
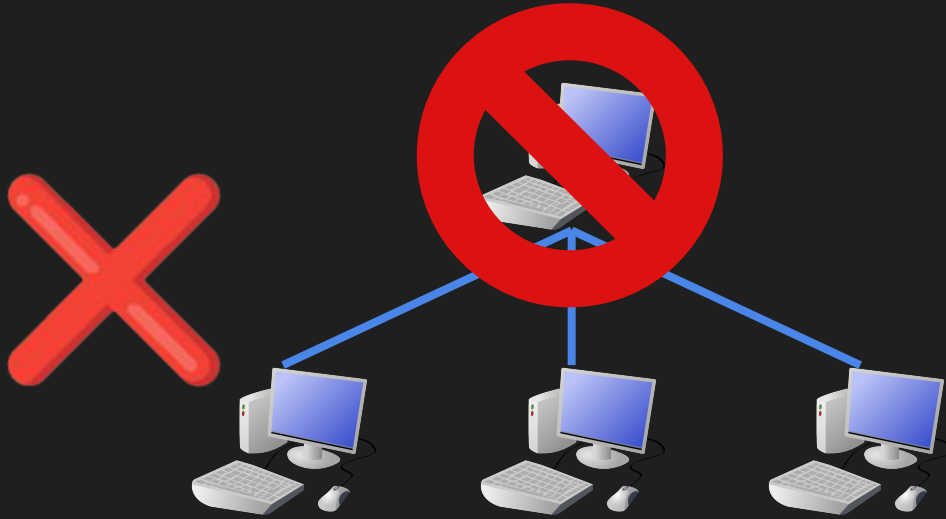
Leader Commit $k=1$



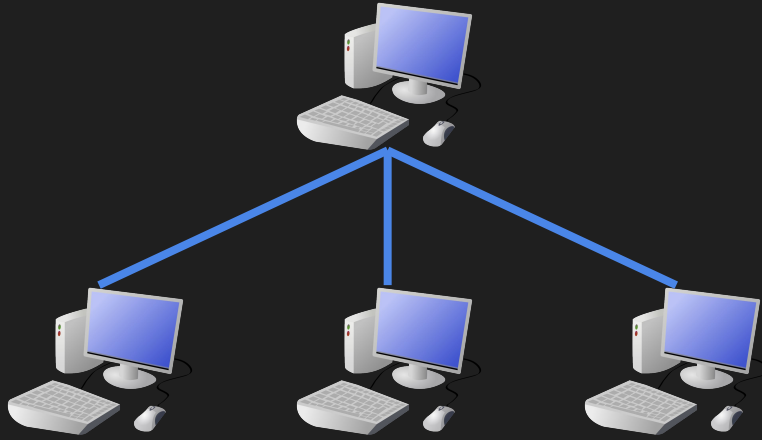
What if a computer goes down?



What if a computer goes down?



Leader Elections



Leader Elections

Candidate



Term: 3

Last log term: 2

Last log index: 5



Candidate Log

Follower



Term: 2

Last log term: 2

Last log index: 4



Follower Log

Other Raft details

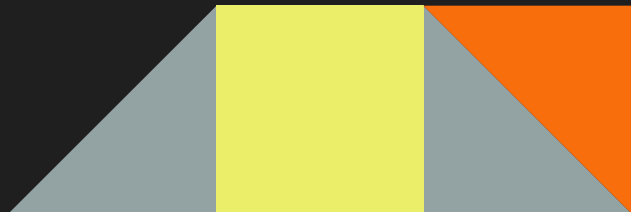
Log prefixes must match

Log Entry Uniqueness

(term and index uniquely identify a log entry)

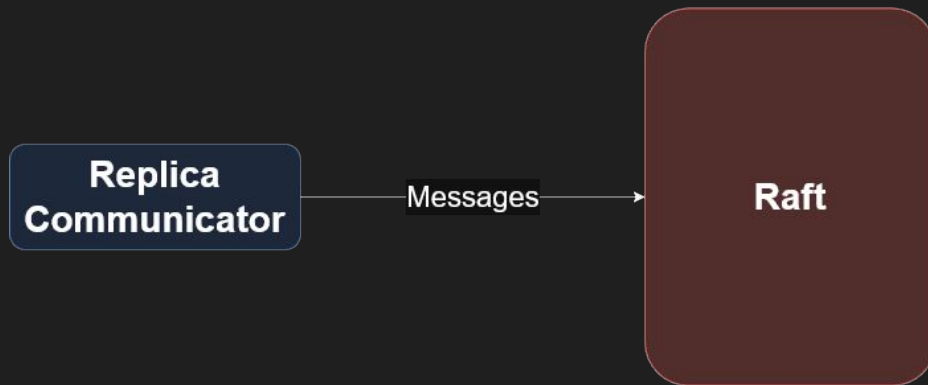
Durable storage

Write-Ahead Log (WAL)

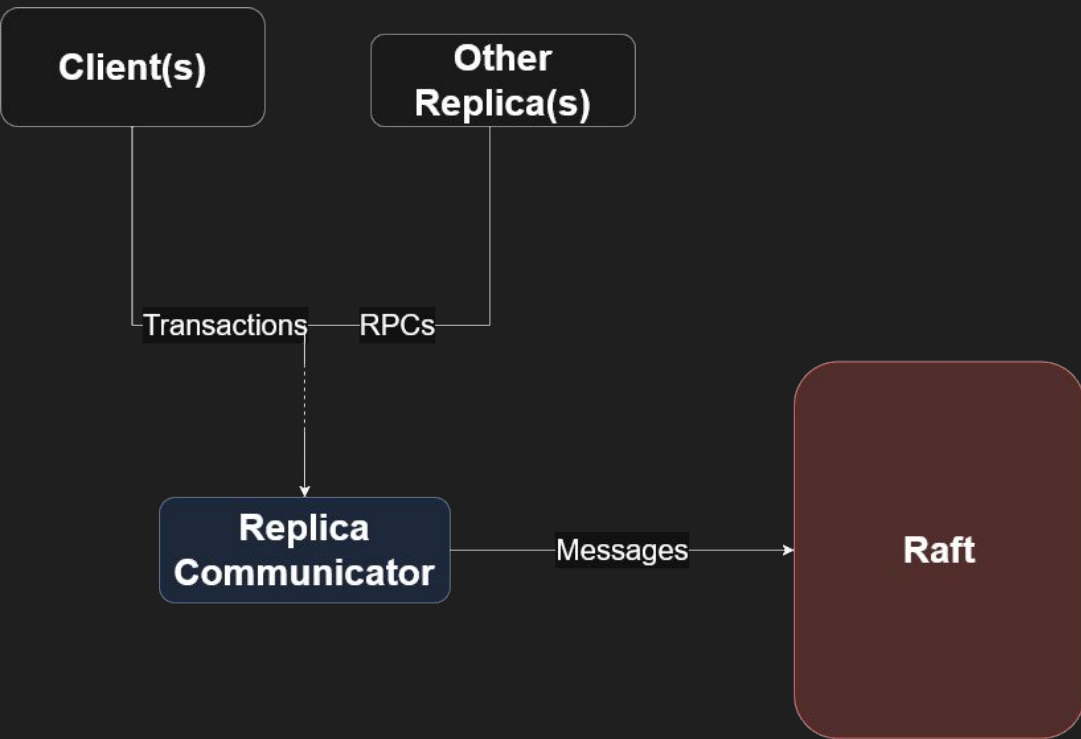


Architecture

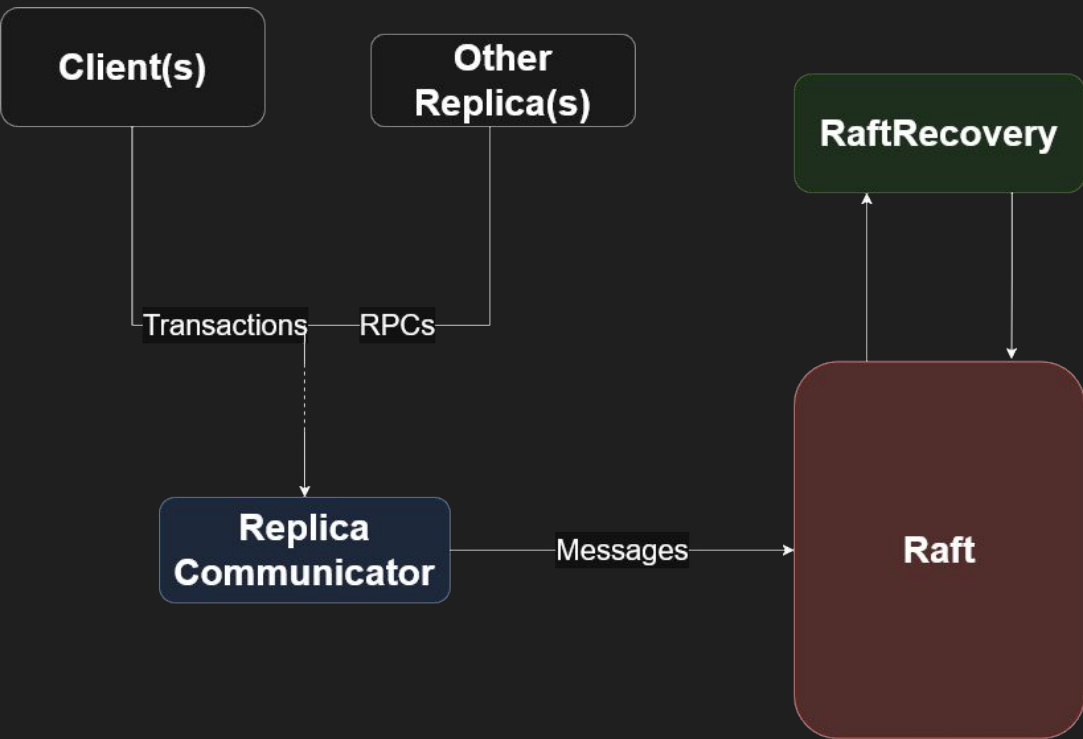
Fitting into ResilientDB



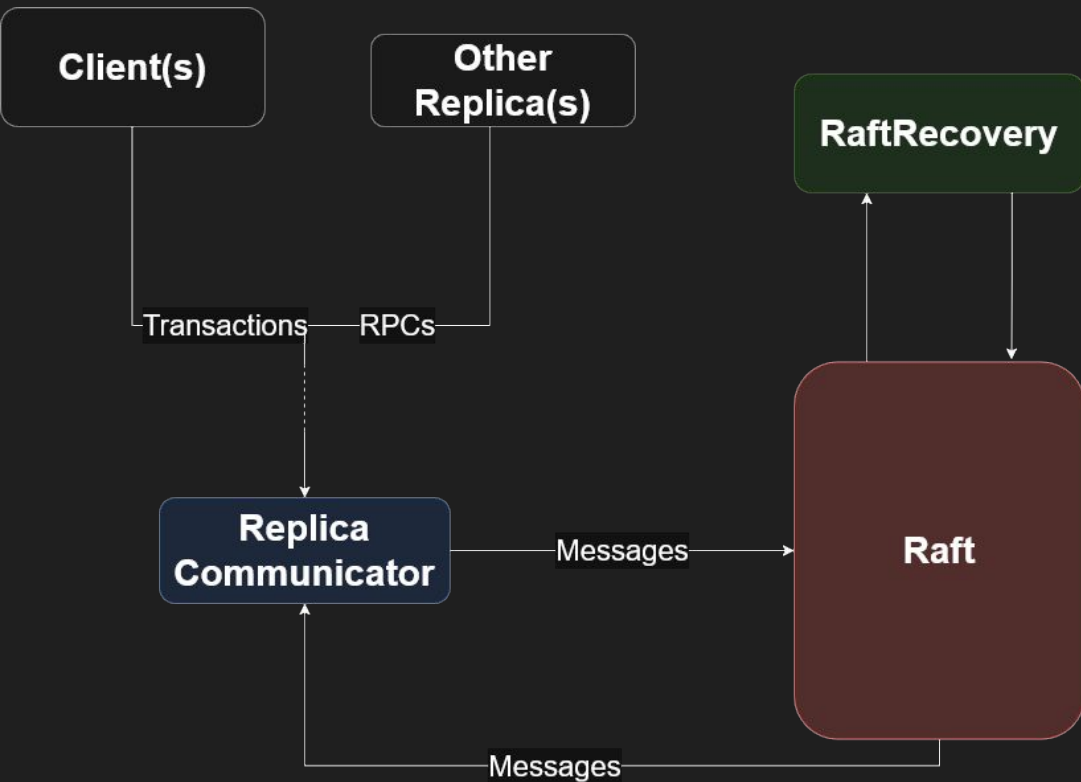
Fitting into ResilientDB



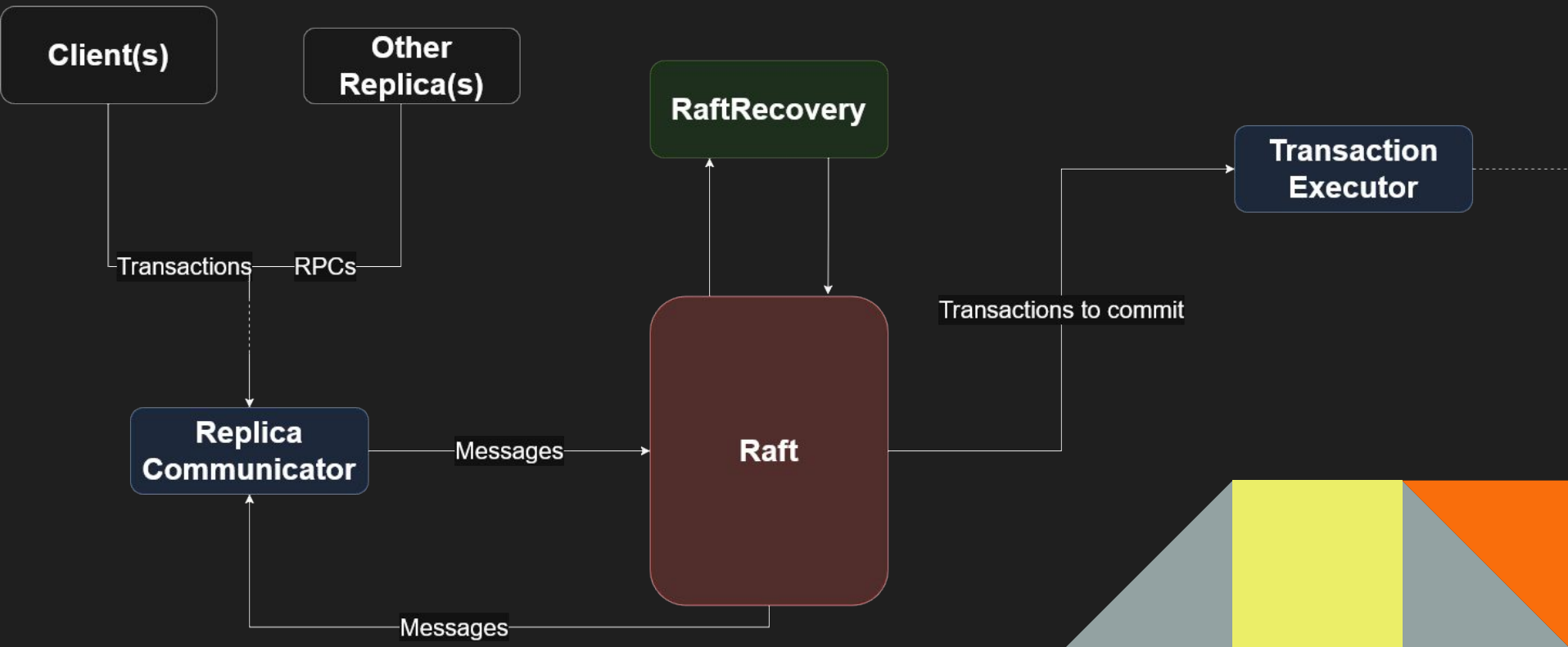
Fitting into ResilientDB



Fitting into ResilientDB



Fitting into ResilientDB



Raft Architecture

Transactions

RPCs and Responses

- AppendEntries
- RequestVote
- InstallSnapshot

RaftRecovery

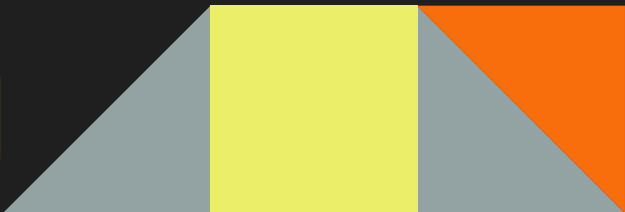
—Messages→

Consensus

—Messages→

Raft

**Leader Election
Manager**

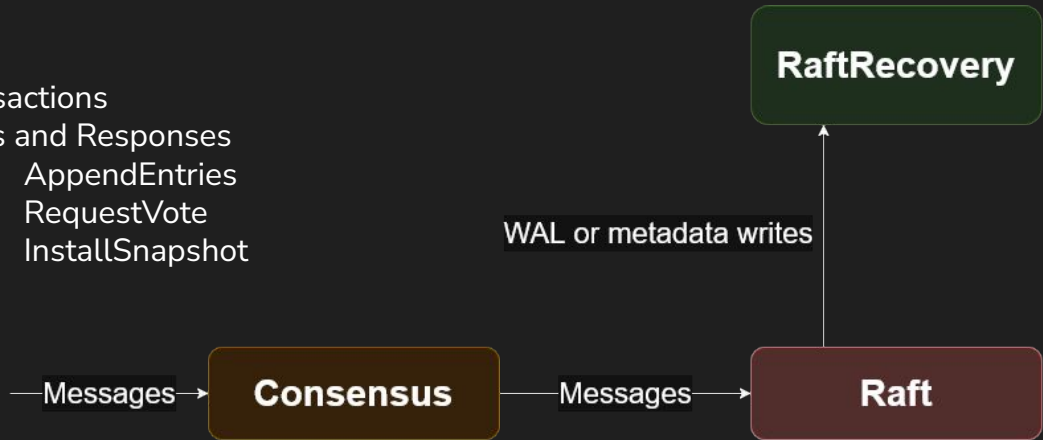


Raft Architecture

Transactions

RPCs and Responses

- AppendEntries
- RequestVote
- InstallSnapshot



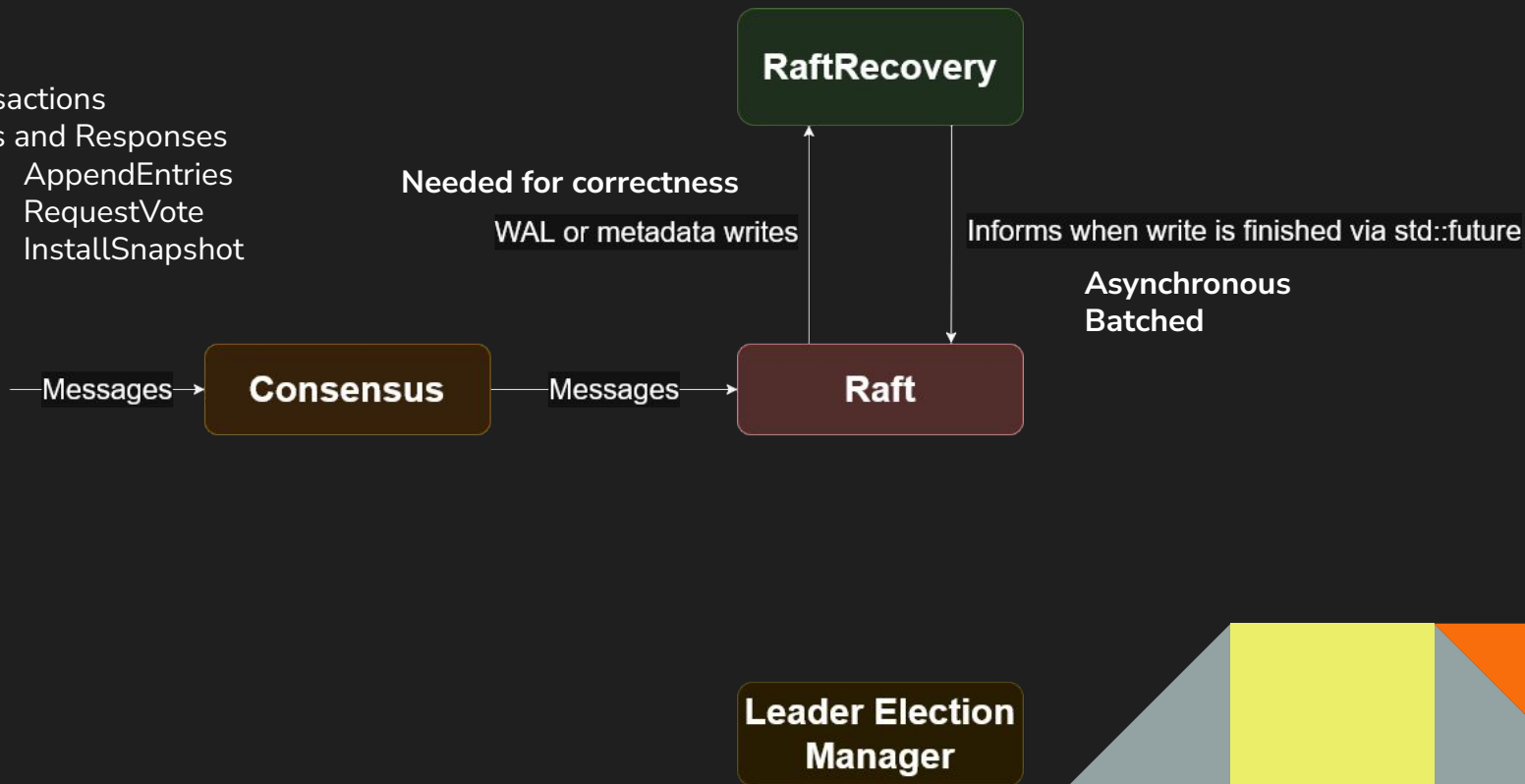
**Leader Election
Manager**

Raft Architecture

Transactions

RPCs and Responses

- AppendEntries
- RequestVote
- InstallSnapshot

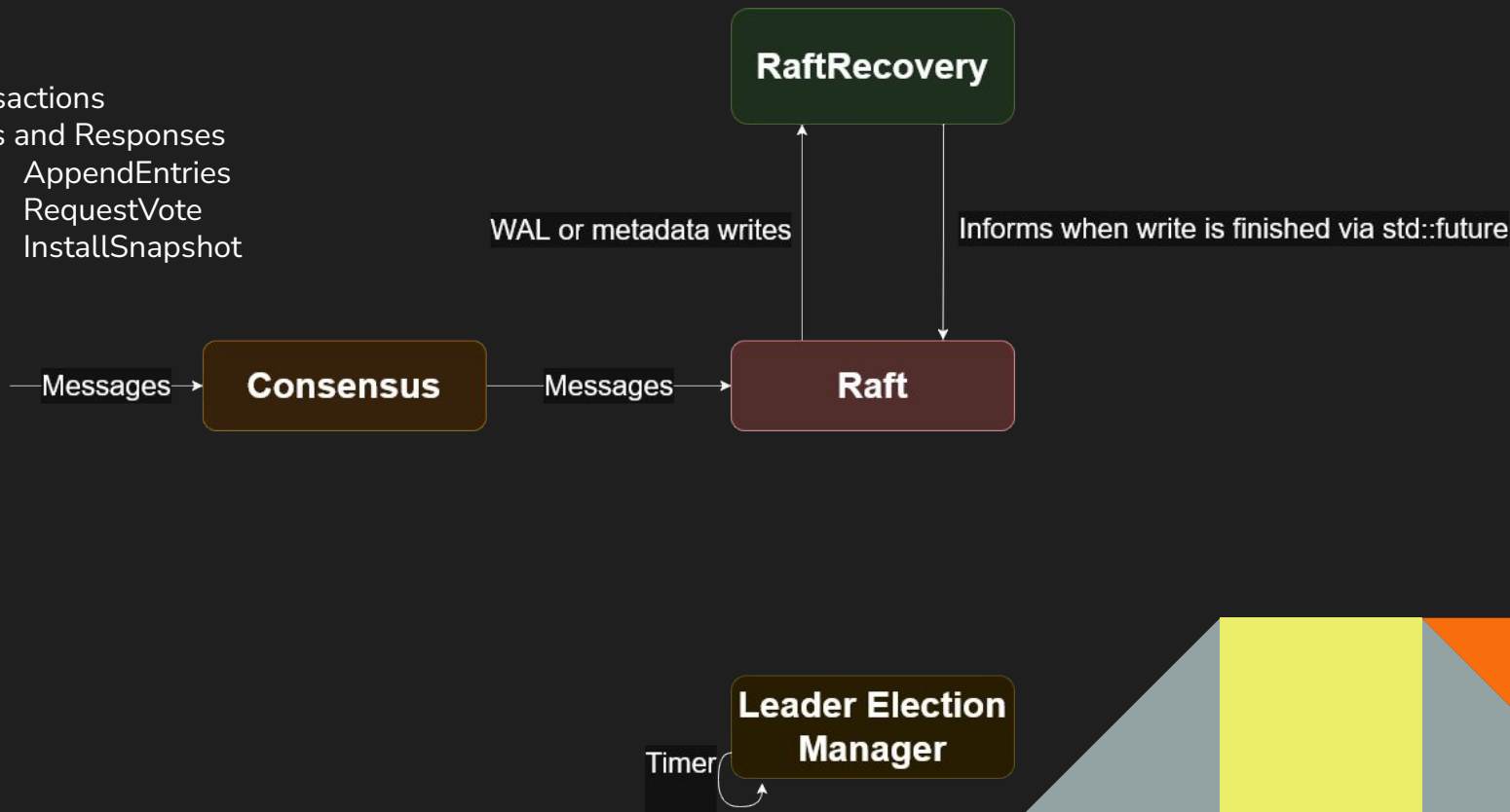


Raft Architecture

Transactions

RPCs and Responses

- AppendEntries
- RequestVote
- InstallSnapshot

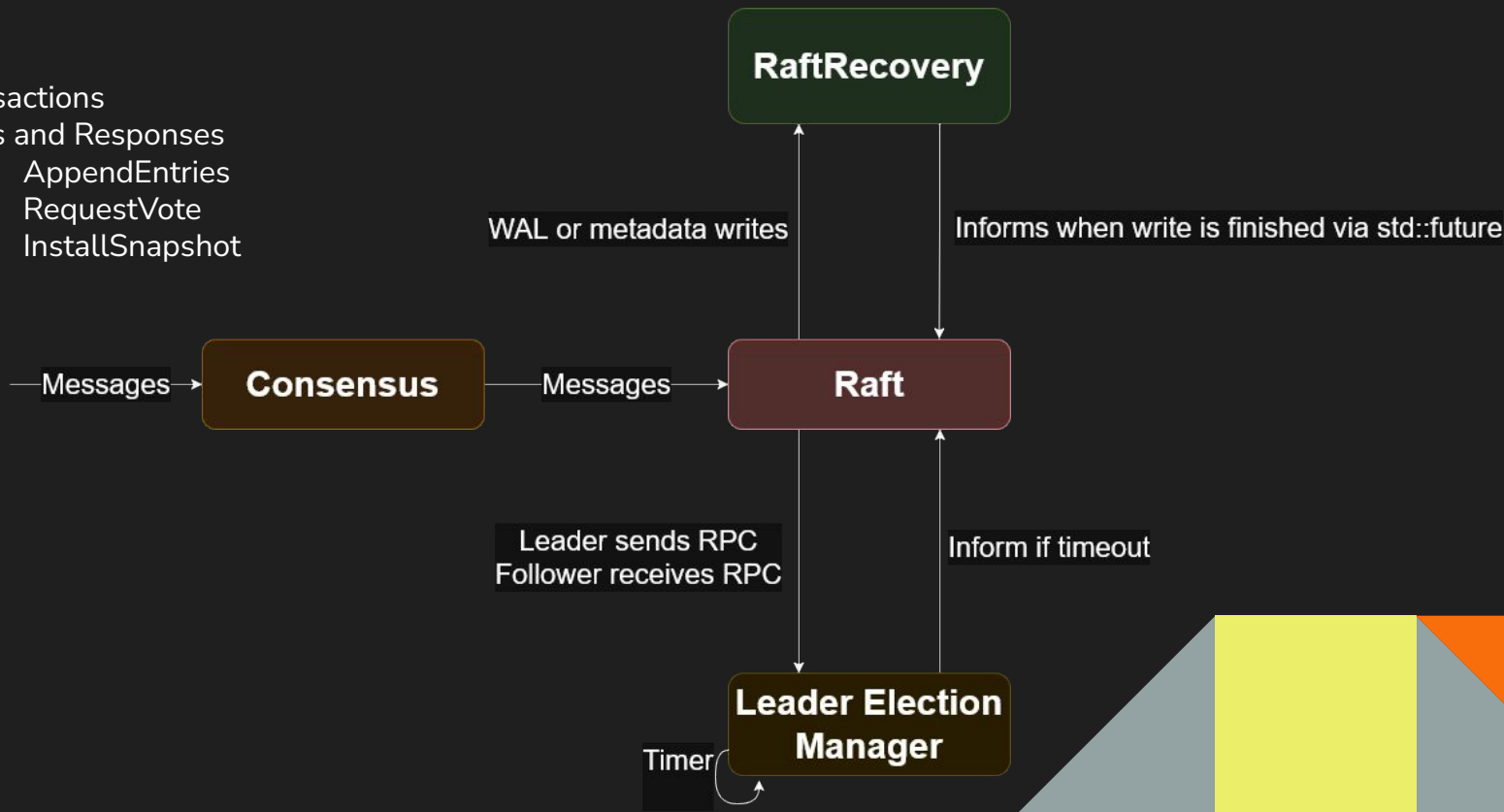


Raft Architecture

Transactions

RPCs and Responses

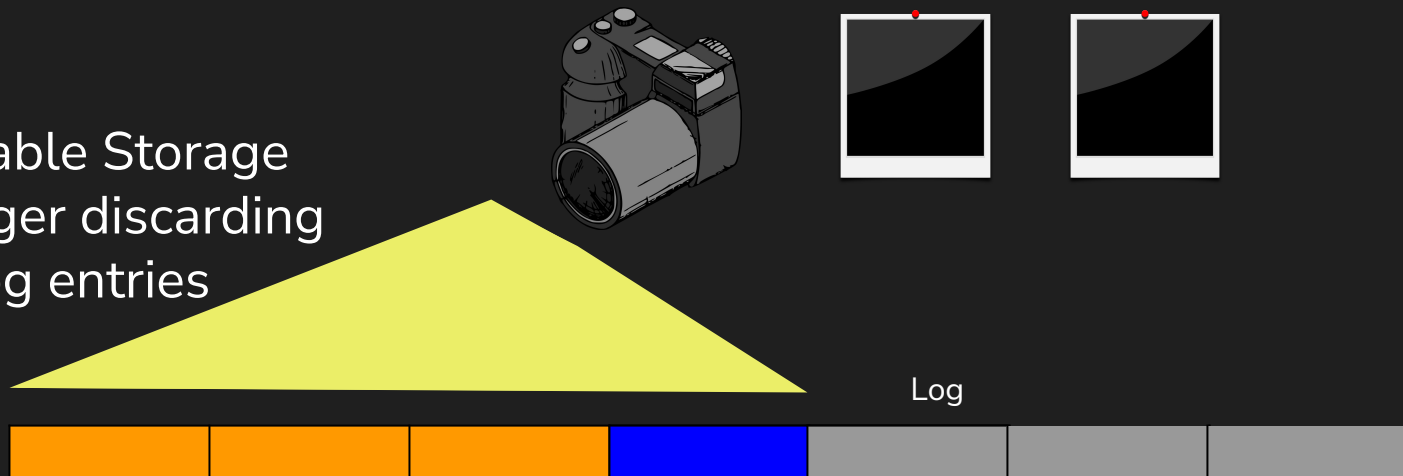
- AppendEntries
- RequestVote
- InstallSnapshot



Recovery

Recovery

1. Durable Storage
2. Trigger discarding of log entries

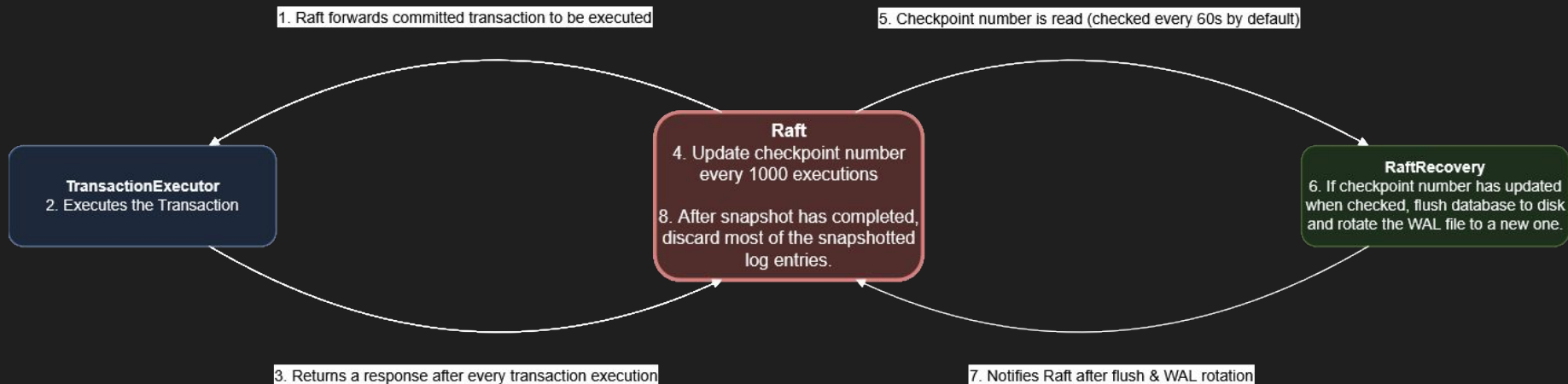


“camera”, tom https://www.publicdomainfiles.com/show_file.php?id=13939271417022

“Polaroid”, smerrell https://www.publicdomainfiles.com/show_file.php?id=13951052419162

Recovery

Raft Snapshot & Log Compaction



Throughput Improvements

Fast Log Backtracking

Index	1	2	3	4	5	6	7
Follower	Term 1			Term 2			
Leader	Term 1			Term 2	Term 3		New Entry

Conflict Term

Conflict Index

LastLogIndex

AE Index

Previous log index: 6

Previous log term: 3

2

4

6

5

Fast Log Backtracking

Index	1	2	3	4	5	6	7
Follower	Term 1			Term 2			
Leader	Term 1			Term 3			New Entry

Conflict Term

2

Conflict Index

4

LastLogIndex

6

AE Index

4

Previous log index: 6

Previous log term: 3

Index	1	2	3	4	5	6	7
Follower	Term 1			Term 2			
Leader	Term 1			Term 2	Term 3		New Entry

Fast Log Backtracking

Index	1	2	3	4	5	6	7
Follower	Term 1			Term 2			
Leader	Term 1			Term 2	Term 3		New Entry

Conflict Term
0

Conflict Index
0

LastLogIndex
5

AE Index
6

Previous log index: 6
Previous log term: 3

Index	1	2	3	4	5	6	7
Follower	Term 1			Term 2			
Leader	Term 1			Term 2	Term 3		New Entry

Index	1	2	3	4	5	6	7
Follower	Term 1			Term 2			
Leader	Term 1			Term 2			New Entry

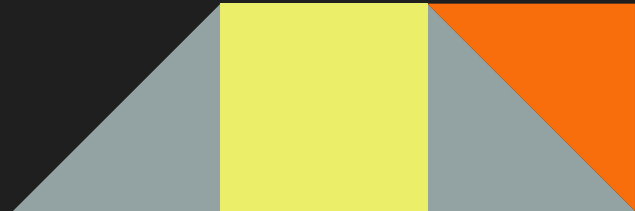
Which entries should be sent?

No pipelining

Simple

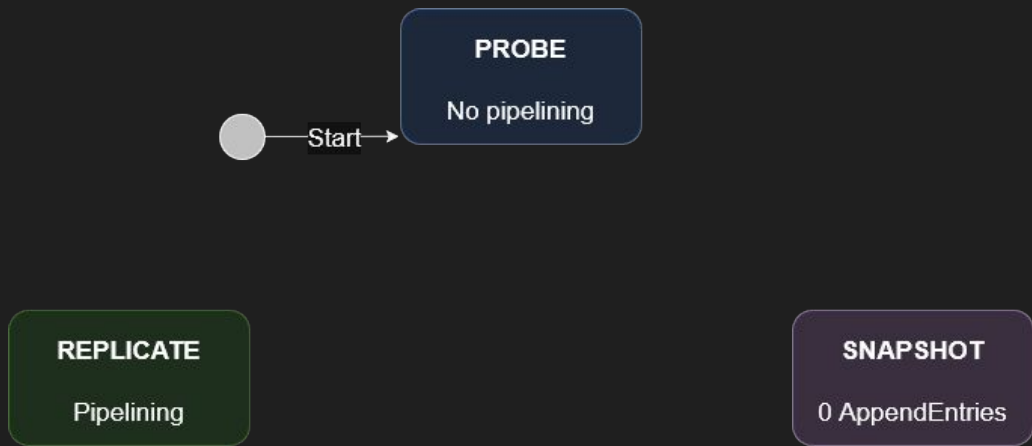
With pipelining

Less simple

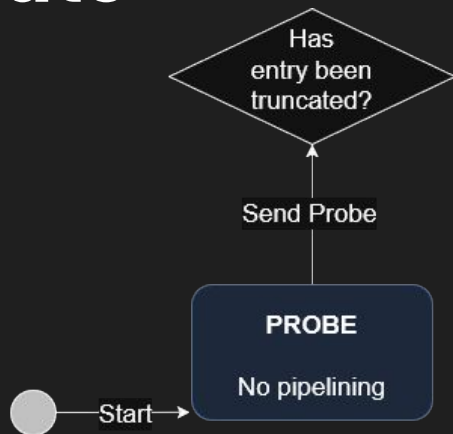


Follower State

*heartbeats can always be sent



Follower State



*heartbeats can always be sent

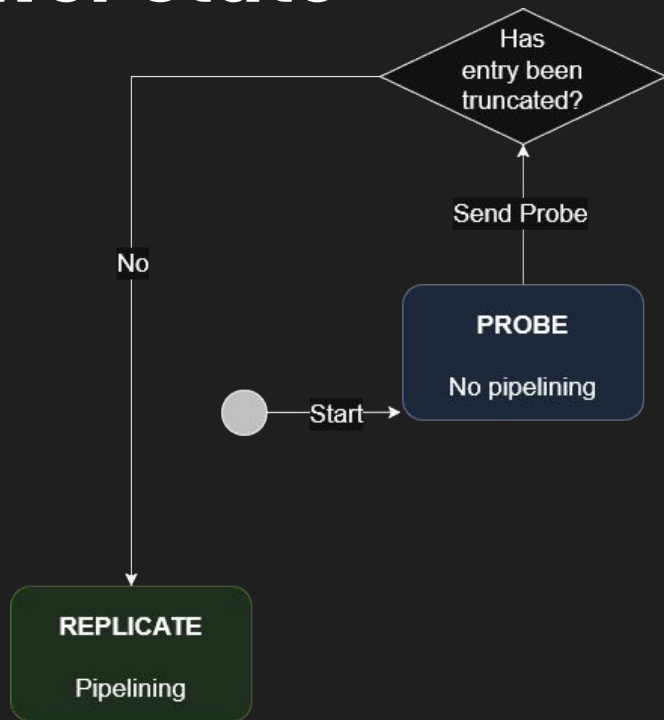
REPLICATE

Pipelining

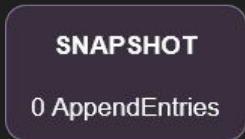
SNAPSHOT

0 AppendEntries

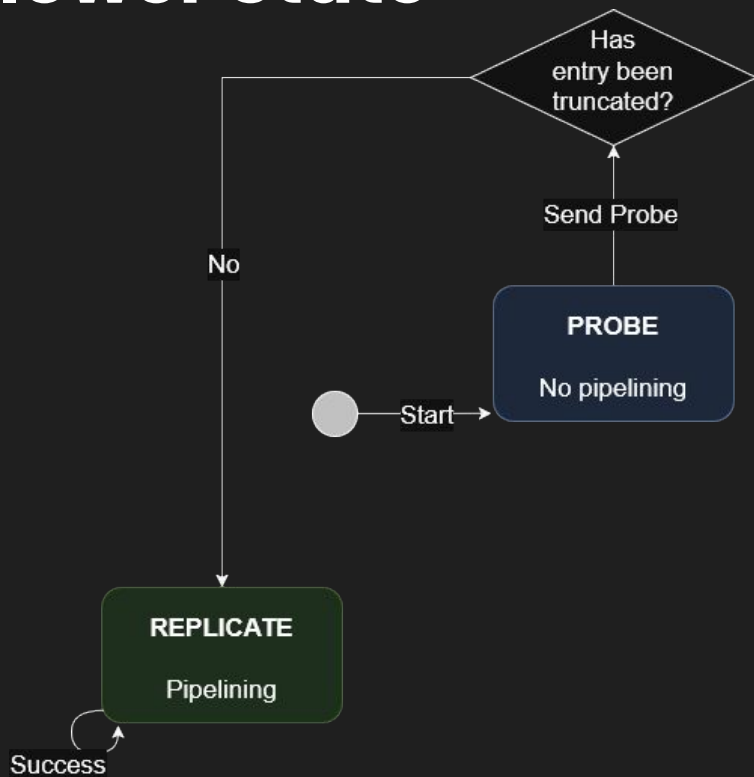
Follower State



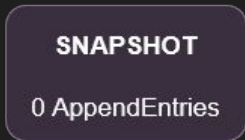
*heartbeats can always be sent



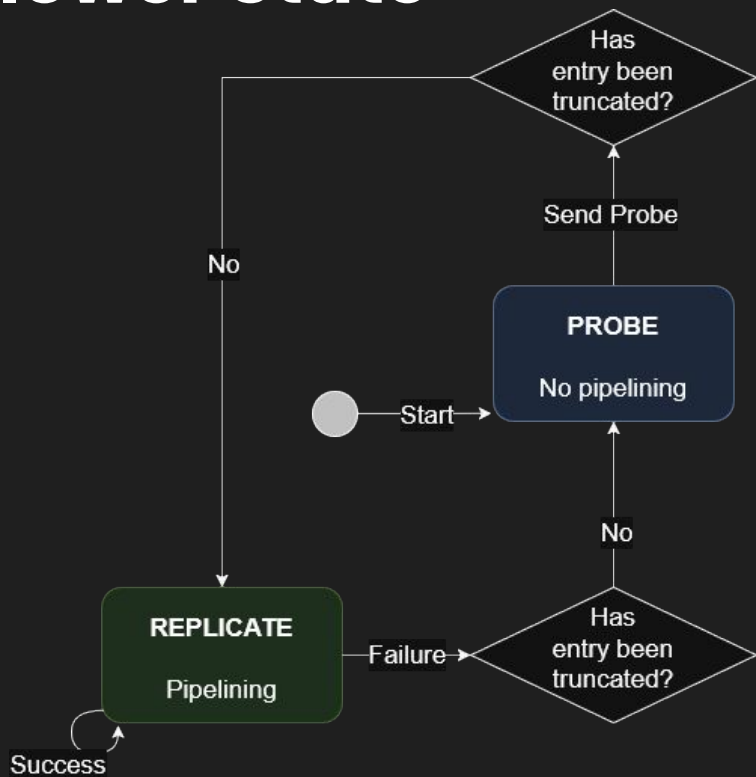
Follower State



*heartbeats can always be sent



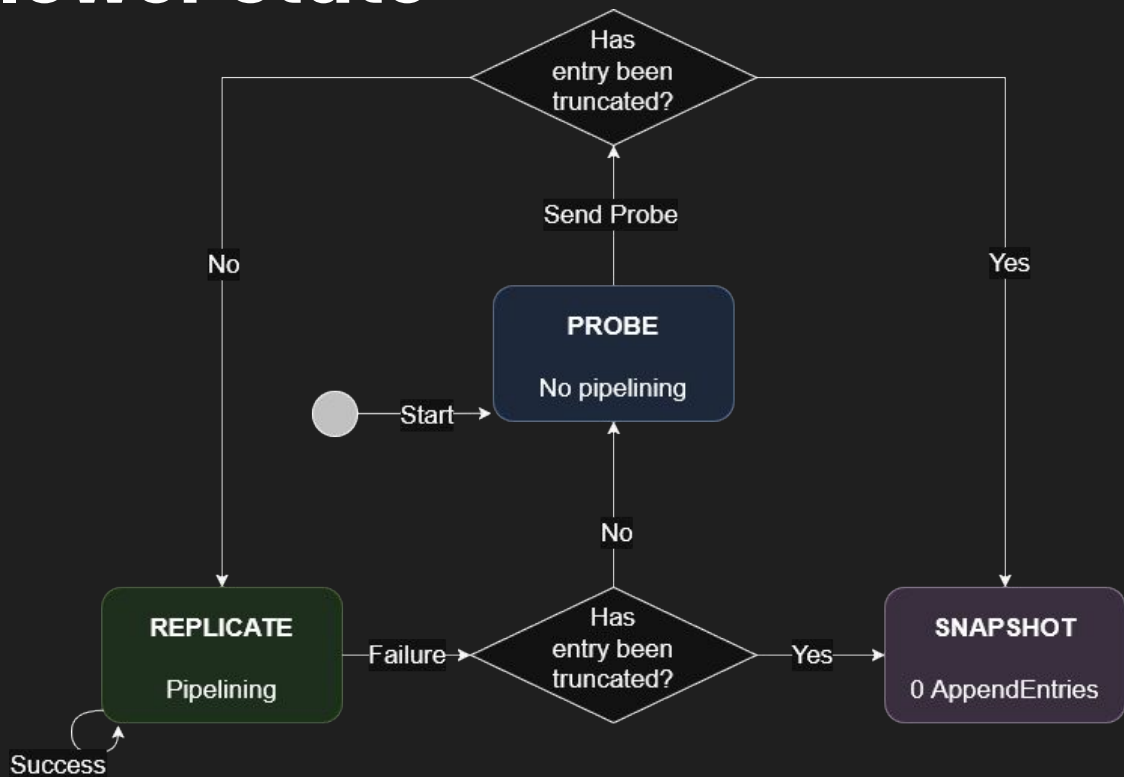
Follower State



*heartbeats can always be sent

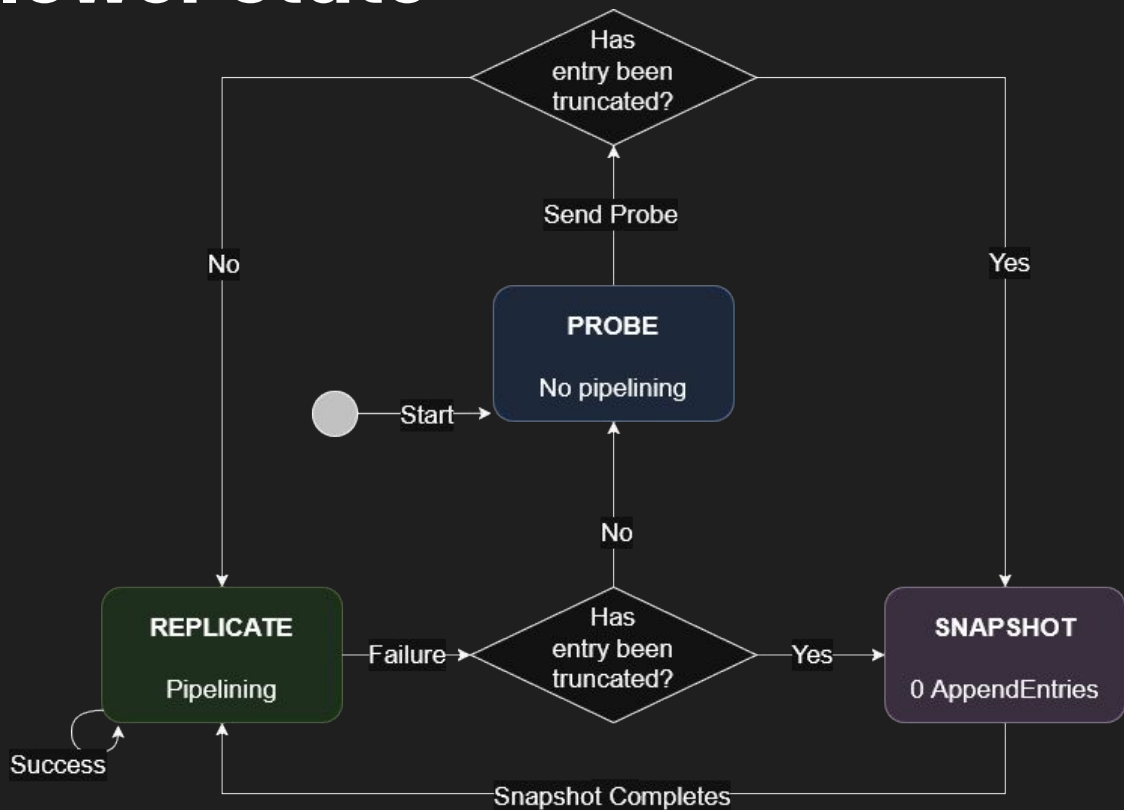
SNAPSHOT
0 AppendEntries

Follower State



*heartbeats can always be sent

Follower State



*heartbeats can always be sent

Evaluation

Cloudlab

d430 computer

Intel Xeon Processor E5-2630v3 2.4 GHz CPU

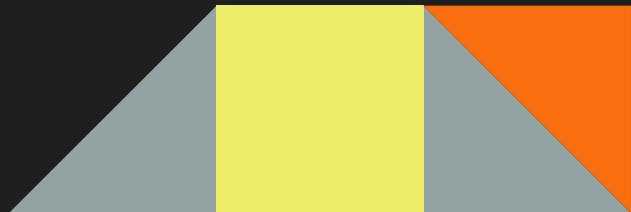
64 GB RAM.

Local network

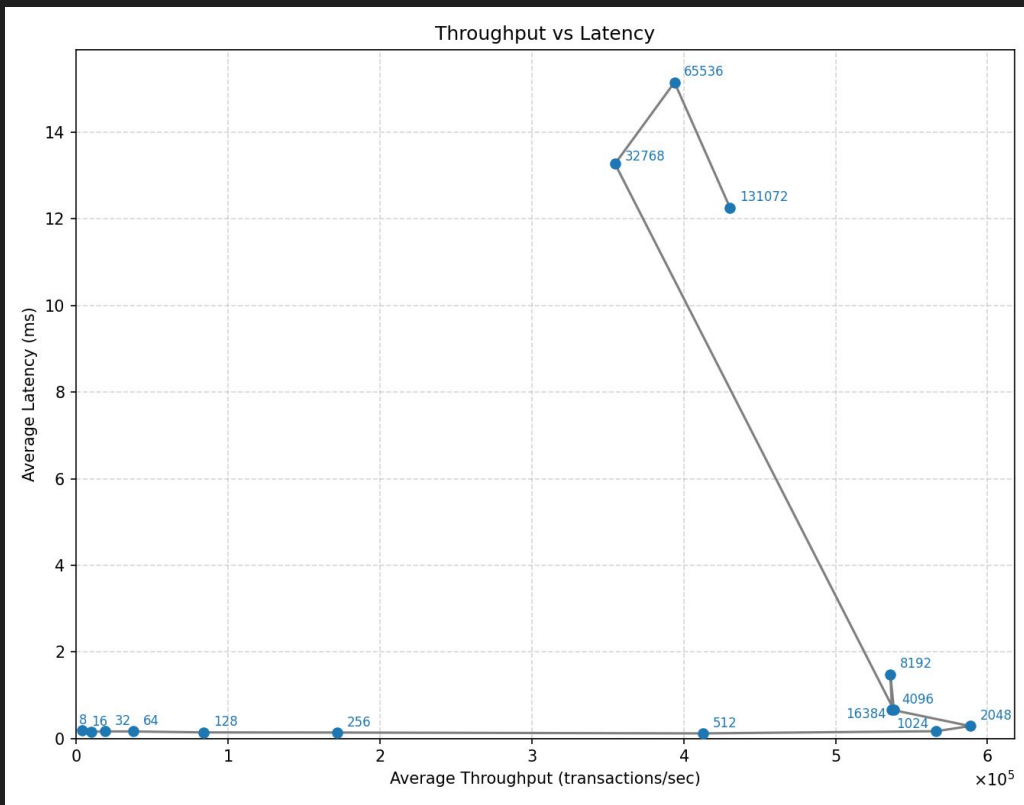
Duration: 120 seconds

Measuring success notifications to client

Median of 3 based on average throughput



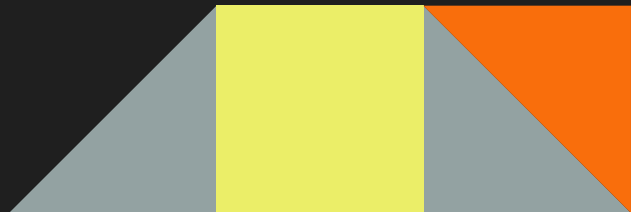
Varying max in-flight transactions



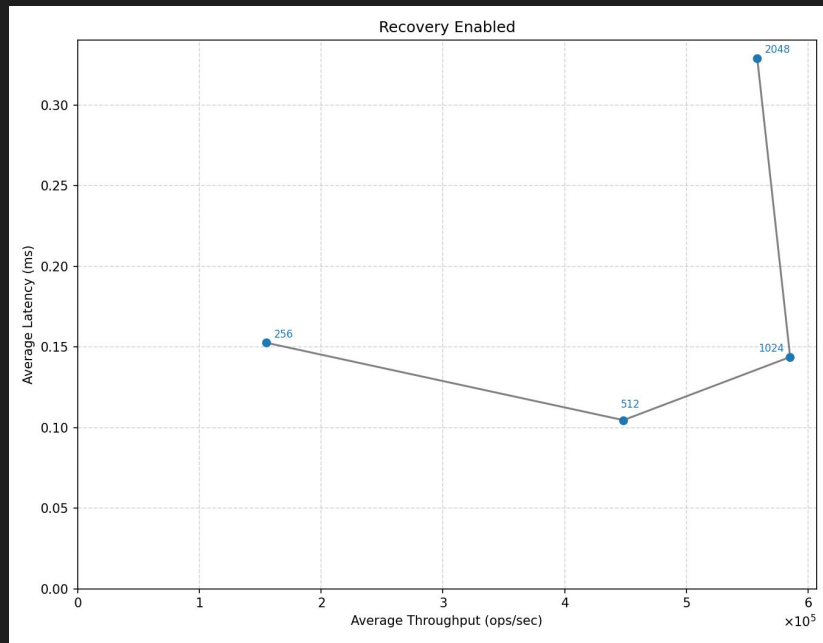
4 Raft nodes +
1 Client

Comparison to Hotstuff-1

Metric	Raft	HotStuff-1
Maximum in-flight transactions from client	2048	3
Average throughput (transactions/second)	588,596	104,080
Average latency (ms)	0.288	2.611



Recovery



Metric

Recovery

No Recovery

No Recovery

Maximum in-flight transactions from client

1024

2048

1024

Average throughput (transactions/second)

585,031

588,596

566,114

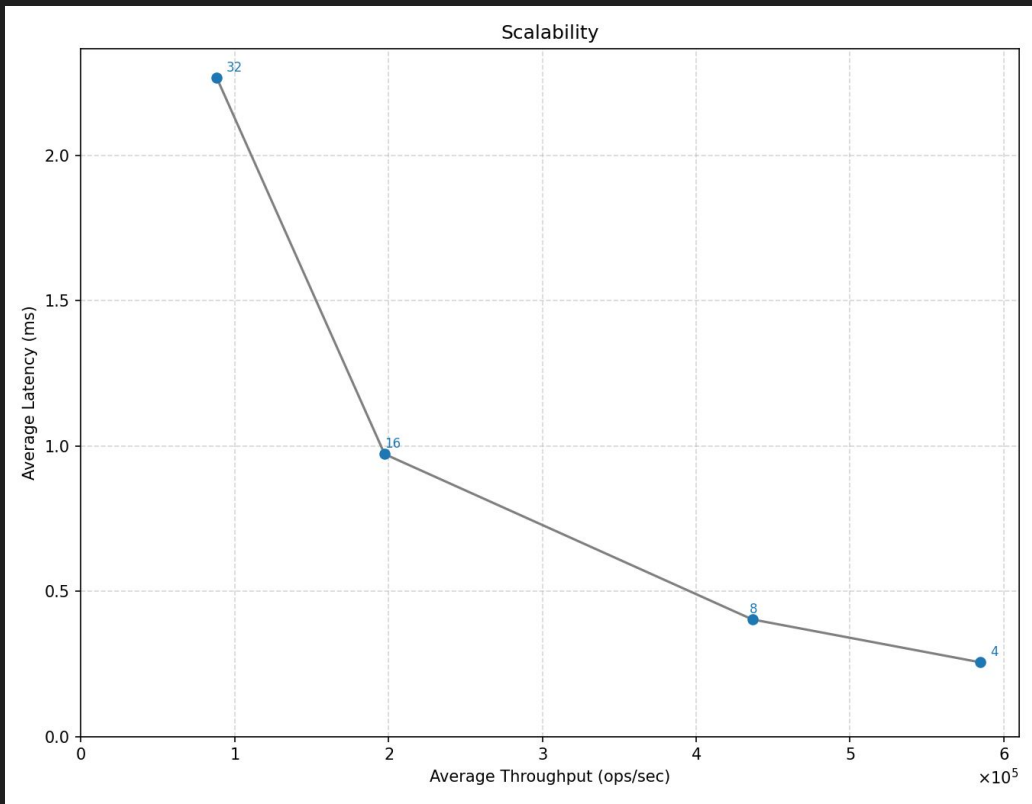
Average latency (ms)

0.144

0.288

0.169

Scalability



Conclusion

Persistence

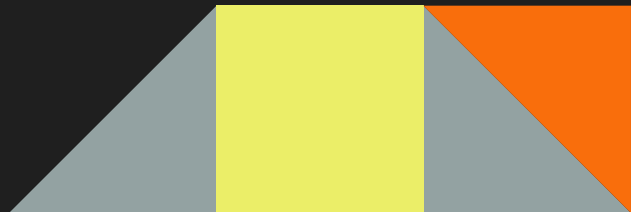
Snapshots

Accelerated Log backtracking

Multithreading

Acknowledgements

Jim Brower, Nachiket Subbaraman, Vinoth Gopikrishnan, and Yuhua Huang



References

S. Gilbert and N. Lynch, “Brewer’s conjecture and the feasibility of consistent, available, partition-tolerant web services,” *SIGACT News*, vol. 33, no. 2, pp. 51–59, Jun. 2002, doi: 10.1145/564585.564601.

etcd-io/raft. (Aug. 21, 2026). Go. etcd-io. Accessed: Aug. 21, 2026. [Online]. Available: <https://github.com/etcd-io/raft>

D. Ongaro and J. Ousterhout, “In search of an understandable consensus algorithm,” in *Proceedings of the 2014 USENIX conference on USENIX Annual Technical Conference*, in USENIX ATC’14. USA: USENIX Association, Jun. 2014, pp. 305–320.

J. Gjengset <jon@thesquareplanet.com>, “Students’ Guide to Raft.” Accessed: Aug. 21, 2026. [Online]. Available: <https://thesquareplanet.com/blog/students-guide-to-raft/>

“Illustration of a computer”, DTRave https://www.publicdomainfiles.com/show_file.php?id=13489758017633

“camera”, tom https://www.publicdomainfiles.com/show_file.php?id=13939271417022

“Polaroid”, smerrell https://www.publicdomainfiles.com/show_file.php?id=13951052419162

